데이터베이스(SQL)

8장. SQL 서브쿼리(Subquery)

학습 목표

1) 서브쿼리의 개념과 읽는 흐름을 설명할 수 있다.
2) 단일 행 서브쿼리와 다중 행 서브쿼리를 구분할 수 있다.
3) IN, EXISTS, NOT EXISTS를 사용할 수 있다.
4) NOT IN의 NULL 문제를 설명하고 회피할 수 있다.
5) 상관 서브쿼리를 이해할 수 있다.
6) 서브쿼리를 사용할 위치별로 구분할 수 있다.

공식 문서: 서브쿼리 표현식

전체 흐름

1) 서브쿼리가 필요한 이유 → 2) 단일 행 / 다중 행 → 3) IN과 NOT IN → 4) EXISTS와 상관 서브쿼리

5) 인라인뷰 / 스칼라 서브쿼리

01 서브쿼리의 기초

서브쿼리란?

서브쿼리(Subquery)는 SQL 문 안에 포함된 또 다른 SELECT 문이다.

구조

```
SELECT * -- 메인쿼리(외부쿼리)
FROM products
WHERE unit_price > (
  SELECT AVG(unit_price) -- 서브쿼리(내부쿼리)
  FROM products
);
```

용어 정리

용어	의미
메인쿼리 / 외부쿼리	바깥쪽 SELECT
서브쿼리 / 내부쿼리	괄호 안 SELECT

서브쿼리는 왜 필요할까?

1) 평균 단가보다 비싼 제품은?
2) 가장 비싼 제품은?
3) 주문 이력이 없는 고객은?
4) 평균보다 많이 주문한 고객은?

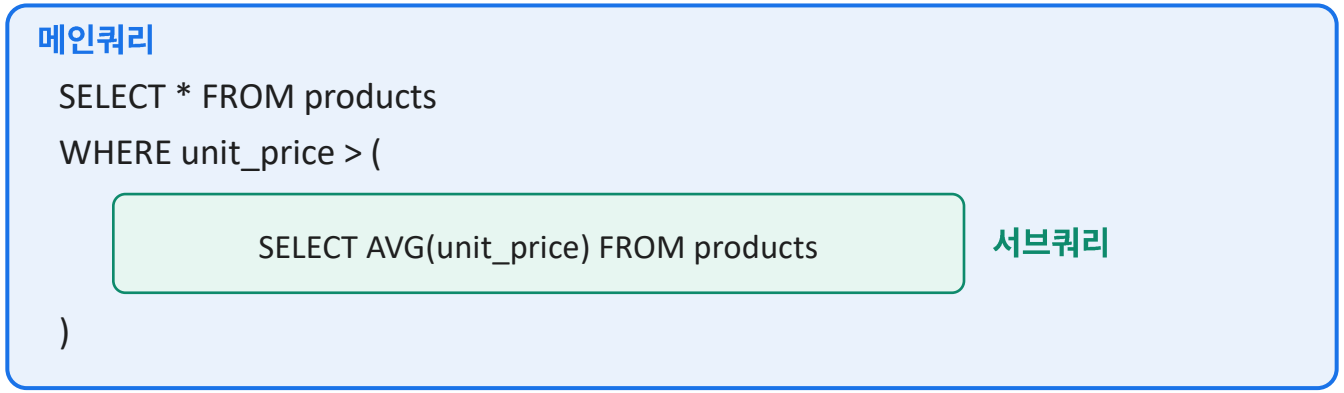
문제 상황 — “평균 단가보다 비싼 제품”을 찾으려면 먼저 평균을 알아야 한다.

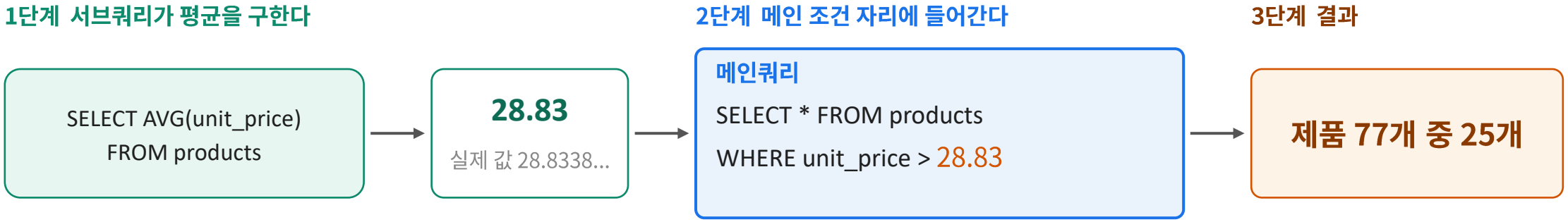
```
-- 1단계: 평균 계산
SELECT AVG(unit_price) FROM products;
-- 결과: 28.8338... (소수점 둘째 자리로 반올림하면 28.83)
```

```
-- 2단계: 그 값을 조건에 사용
SELECT * FROM products WHERE unit_price > 28.83;
```

두 번 실행해야 하고, 데이터가 바뀌면 평균값도 다시 계산해야 한다.

서브쿼리의 논리적 흐름


메인쿼리
SELECT * FROM products
WHERE unit_price > (
SELECT AVG(unit_price) FROM products
서브쿼리
)


1단계 서브쿼리가 평균을 구한다
SELECT AVG(unit_price)
FROM products
28.83
실제 값 28.8338...
2단계 메인 조건 자리에 들어간다
메인쿼리
SELECT * FROM products
WHERE unit_price > 28.83
3단계 결과
제품 77개 중 25개

서브쿼리 사용 위치

위치	명칭	용도
WHERE	조건 서브쿼리	조건 비교
FROM	인라인 뷰 / 파생 테이블	테이블처럼 사용
SELECT	스칼라 서브쿼리	컬럼처럼 값 하나 반환
HAVING	그룹 조건 서브쿼리	그룹 필터링

02 단일 행과 다중 행 서브쿼리

단일 행 서브쿼리

실행 결과로 하나의 행을 반환하는 서브쿼리이다. 반환된 열의 수에 따라 하나의 값 또는 여러 값을 포함할 수 있다

단일 값 비교에 사용하는 연산자

연산자	의미
=	같다
<>	다르다
>	크다
<	작다
>=	크거나 같다
<=	작거나 같다

단일 행 서브쿼리 예제

가장 비싼 제품

```
SELECT
    product_name,
    unit_price
FROM products
WHERE unit_price = (
    SELECT MAX(unit_price) FROM products
);
```

평균 초과 제품

```
SELECT
    product_name,
    unit_price
FROM products
WHERE unit_price > (
        SELECT AVG(unit_price) FROM products
        )
ORDER BY unit_price DESC;
```

다중 행 서브쿼리

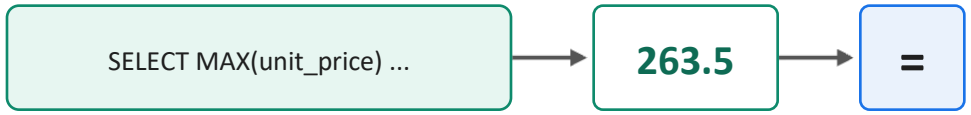
실행 결과로 여러 행을 반환하는 서브쿼리로, IN, ANY, ALL 등의 연산자와 함께 활용할 수 있다.

```
-- 오류 발생
SELECT *
FROM customers
WHERE customer_id = (
   SELECT customer_id FROM orders -- 830행 (여러 번 주문한 고객은 같은 customer_id가 여러 번 나온다)
);
-- ERROR: more than one row returned by a subquery used as an expression
```

오류: 표현식에 사용된 서브쿼리 결과가 한 행보다 많은 행을 리턴했습니다

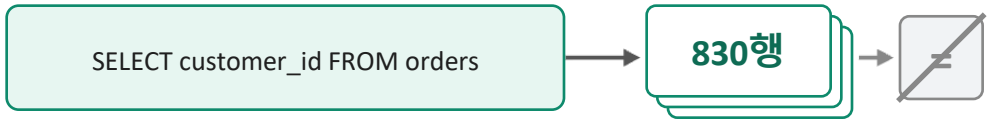
단일 행과 다중 행

단일 행 서브쿼리



값 1개 → = > < 사용 가능

다중 행 서브쿼리



IN·ANY·ALL: 값 비교 / EXISTS: 행 존재

ERROR: more than one row returned by a subquery used as an expression

다중 행 서브쿼리에 쓰는 연산자

연산자	의미
IN	목록에 포함됨
NOT IN	목록에 포함 안 됨
ANY / SOME	하나라도 만족
ALL	모두 만족
EXISTS	서브쿼리 결과에 행이 존재함
NOT EXISTS	서브쿼리 결과에 행이 없음

03 IN과 NOT IN

IN과 서브쿼리

IN: 특정 값이 지정한 값 목록 또는 서브쿼리 결과에 포함되는지 확인하는 SQL 조건 표현식이다.

```
-- 주문 이력이 있는 고객
SELECT
    customer_id,
    company_name
FROM customers
WHERE customer_id IN (
    SELECT customer_id FROM orders
);
```

공식 문서: IN 서브쿼리

IN의 동작 원리 — 주문 이력이 있는 고객 조회

IN은 고객 ID가 서브쿼리 결과에 포함되어 있는지를 확인

① 서브쿼리 결과

orders.customer_id (일부)

customer_id
ALFKI
ALFKI
ANATR
…

② 고객 ID 포함 여부 확인

customers 고객 91명 중 예시

고객 ID	IN 결과
ALFKI	포함
ANATR	포함
FISSA	없음
PARIS	없음

③ 최종 SELECT 결과

customer_id, company_name 조회

고객 ID	회사명 (일부)
ALFKI	Alfreds Futterkiste
ANATR	Ana Trujillo …
…	…

교안 실습 데이터 기준: 89명

NOT IN의 NULL 문제

NOT IN : 특정 값이 지정한 값 목록 또는 서브쿼리 결과에 포함되지 않는지 확인하는 SQL 조건 표현식이다.

```
-- 주문 이력이 없는 고객을 찾으려는 시도
SELECT
   customer_id,
   company_name
FROM customers
WHERE customer_id NOT IN (
   SELECT customer_id FROM orders
);
```

NOT IN의 NULL 함정

Northwind의 orders.customer_id에는 NULL이 없어 이 예제는 정상 동작하지만, NULL이 있을 수 있는 데이터에서는 주의한다.

공식 문서: NOT IN

04 EXISTS와 상관 서브쿼리

EXISTS

서브쿼리의 실행 결과에 행이 하나 이상 존재하는지 확인하여 TRUE 또는 FALSE를 반환하는 SQL 조건 표현식

```
SELECT
    c.customer_id,
    c.company_name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

공식 문서: EXISTS

SELECT 1을 쓰는 이유

```
-- 아래 세 개 모두 동일하게 동작한다
SELECT 1 FROM orders WHERE ...
SELECT * FROM orders WHERE ...
SELECT customer_id FROM orders WHERE ...
```

EXISTS는 **서브쿼리가 행을 반환하는지**를 확인하므로 일반적인 EXISTS 예제에서는 SELECT 목록의 실제 값이 중요하지 않다.

IN vs EXISTS 비교

항목	IN	EXISTS
중심 의미	값이 결과 집합에 포함되는가	조건을 만족하는 행이 존재하는가
오른쪽 결과	한 컬럼의 값 집합	임의의 SELECT 결과 행 존재 여부
NULL 주의	중요	NOT IN과 같은 NULL 목록 함정은 없음
상관 가능 여부	가능	가능

NOT EXISTS

서브쿼리의 실행 결과에 행이 존재하지 않는지 확인하여 TRUE 또는 FALSE를 반환하는 SQL 조건 표현식이다.

한 번도 주문하지 않은 고객

```
-- 한 번도 주문하지 않은 고객
SELECT
    c.customer_id,
    c.company_name
 FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

한 번도 판매되지 않은 제품

```
-- 한 번도 판매되지 않은 제품
SELECT
    p.product_id,
    p.product_name
FROM products p
WHERE NOT EXISTS (
    SELECT 1
    FROM order_details od
    WHERE od.product_id = p.product_id
);
```

상관 서브쿼리(Correlated Subquery)

비상관 서브쿼리 (일반)
외부 쿼리의 컬럼을 참조하지 않고 독립적으로 작성·실행할 수 있는 서브쿼리이다.

```
-- 서브쿼리가 독립적으로 실행 가능
SELECT * FROM products
WHERE unit_price > (
   SELECT AVG(unit_price) FROM products
);
```

상관 서브쿼리
외부 쿼리의 컬럼을 참조하여 외부 쿼리와 연관된 조건이나 계산을 수행하는 서브쿼리이다.

```
-- 서브쿼리가 메인쿼리의 c를 참조
SELECT * FROM customers c
WHERE EXISTS (
   SELECT 1 FROM orders o
   WHERE o.customer_id = c.customer_id -- c는 메인쿼리 것
);
```

서브쿼리만 떼어내면 c를 알 수 없어 실행되지 않는다.

구분	비상관 서브쿼리	상관 서브쿼리
바깥 쿼리 값 참조	없음	있음
서브쿼리만 독립 실행	가능	바깥 참조를 제거하지 않으면 불가능

상관 서브쿼리를 따로 실행하면

```
-- 서브쿼리만 실행하면 오류
SELECT 1 FROM orders o
WHERE o.customer_id = c.customer_id;
-- ERROR: missing FROM-clause entry for table "c"
```

오류: 테이블 "c"에 FROM 절이 빠져 있습니다.

상관 서브쿼리 예제

```
-- 같은 카테고리 평균보다 비싼 제품
SELECT
    p.product_name,
    p.category_id,
    p.unit_price
FROM products p
WHERE p.unit_price > (
    SELECT AVG(p2.unit_price)
    FROM products p2
    WHERE p2.category_id = p.category_id -- 상관 조건
)
ORDER BY p.category_id, p.unit_price DESC, p.product_name;
```

상관 서브쿼리의 평가 — 카테고리별 평균 가격 비교

바깥 행의 카테고리 번호를 안쪽 서브쿼리에서 참조하여, 각 제품을 자기 카테고리의 평균과 비교한다.

① 전체 SQL — 바깥쪽 p / 안쪽 p2

```
SELECT
    p.product_name,
    p.category_id,
    p.unit_price
FROM products AS p
WHERE p.unit_price > (
    SELECT AVG(p2.unit_price)
    FROM products AS p2
    WHERE p2.category_id = p.category_id
);
```

② 카테고리별 평균 가격 (예시)

카테고리 1 — 음료	37.98
카테고리 2 — 조미료	23.06

③ 제품별 가격 비교 (일부)

카테고리	제품명	가격	평균	판정
1	Chai	18.00	37.98	제외
1	Côte de Blaye	263.50	37.98	포함
1	Ipoh Coffee	46.00	37.98	포함
2	Aniseed Syrup	10.00	23.06	제외
2	Grandma’s Boysenberry Spread	25.00	23.06	포함
2	Northwoods Cranberry Sauce	40.00	23.06	포함

각 제품은 전체 평균이 아니라, 자신이 속한 카테고리의 평균 가격과 비교된다.

05 FROM 절과 SELECT 절 서브쿼리

FROM 절 서브쿼리 (파생 테이블)

FROM 절의 서브쿼리는 테이블처럼 동작한다. 파생 테이블(Derived Table) 또는 인라인 뷰라고 부른다.

문법

```
SELECT ...
FROM (
        SELECT ...
) AS 별칭 -- 별칭 (PostgreSQL 16부터 생략 가능, 작성 권장)
```

공식 문서: 서브쿼리 (FROM 절)

별칭을 쓰는 이유

이유	설명
컬럼 참조	t.avg_price처럼 한정해서 참조하려면 별칭이 필요하다
여러 FROM 항목 구분	별칭을 붙이면 t.컬럼처럼 어느 결과의 컬럼인지 명확히 쓸 수 있다
호환성	다른 데이터베이스에서는 별칭이 필수이므로 호환을 위해 쓴다

활용 예 - 집계 결과 재조회

```
SELECT
   t.category_id, -- 카테고리 번호
   t.avg_price -- 평균 단가
FROM (
   SELECT
      category_id,
      AVG(unit_price) AS avg_price -- 평균 단가
   FROM products
   GROUP BY category_id
) AS t
WHERE t.avg_price > 30
ORDER BY t.avg_price DESC;
```



SELECT 절 서브쿼리 (스칼라 서브쿼리)

SELECT 절에 서브쿼리를 넣어 컬럼처럼 사용한다.

스칼라 서브쿼리의 정확한 조건

스칼라 서브쿼리는 **1개 컬럼,최대 1개 행**을 반환해야 한다.

1) 1행이면 그 값을 사용한다.

2) 0행이면 결과는 NULL이다.

3) 2행 이상이면 오류가 난다.

4) 컬럼을 2개 이상 반환하는 형태도 스칼라 서브쿼리로 사용할 수 없다.

공식 문서: 스칼라 서브쿼리

스칼라 서브쿼리 예제

```
SELECT
    product_name, -- 제품명
    unit_price, -- 단가
    (SELECT AVG(unit_price) FROM products) AS overall_avg, -- 전체 평균 단가
    ROUND(
        (unit_price - (SELECT AVG(unit_price) FROM products))::NUMERIC, 2
    ) AS diff_from_avg -- 평균과의 차이
FROM products
ORDER BY unit_price DESC, product_id -- 단가가 같으면 제품 번호 순
LIMIT 10;
```

상관 스칼라 서브쿼리

외부 쿼리의 컬럼을 참조하면서, 각 평가 결과를 단일 값으로 반환하는 스칼라 서브쿼리이다.

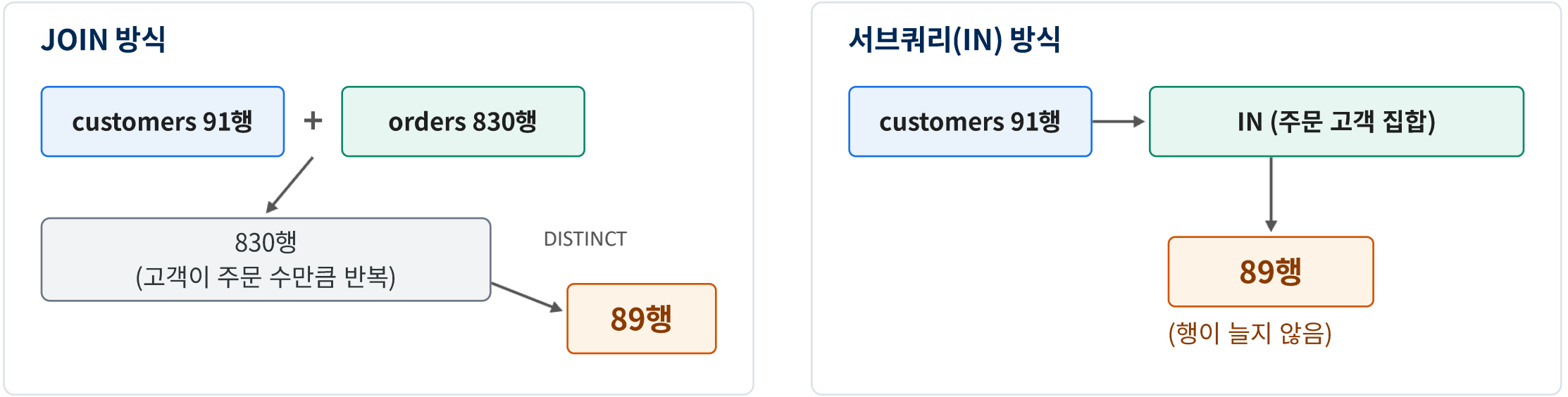
```
-- 각 고객의 주문 건수를 컬럼으로
SELECT
   c.customer_id, -- 고객 코드
    c.company_name, -- 회사명
   (
        SELECT COUNT(*)
        FROM orders o
        WHERE o.customer_id = c.customer_id
    ) AS order_count -- 주문 건수
FROM customers c
ORDER BY order_count DESC, c.customer_id -- 건수가 같으면 고객 코드 순
LIMIT 10;
```

같은 결과를 두 방법으로

```
-- 서브쿼리 방식
SELECT customer_id, company_name
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);

-- JOIN 방식
SELECT DISTINCT c.customer_id, c.company_name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id;
```

JOIN과 IN의 행 수 차이



JOIN은 1:N에서 행이 늘어나므로 DISTINCT가 필요하고, IN은 고객 행이 늘지 않는다

JOIN과 서브쿼리 비교표

항목	JOIN	서브쿼리
주로 쓰는 상황	서로 다른 테이블의 행을 연결	조건 판단, 집계값 비교, 중간 결과 사용
결과 컬럼	연결한 FROM 항목의 컬럼을 함께 사용하기 쉬움	위치에 따라 다름: 스칼라 서브쿼리는 값 추가, FROM 서브쿼리는 테이블처럼 사용
가독성	관계 연결이 중심이면 읽기 쉬움	단계적 조건이나 계산을 표현하기 좋음
행 수	관계에 따라 증가 가능	EXISTS/스칼라/FROM 등 형태에 따라 다름

JOIN과 서브쿼리는 서로 완전히 대립하는 문법이 아니다.
서브쿼리 안에서도 JOIN을 사용할 수 있고, FROM 서브쿼리를 바깥 쿼리와 다시 JOIN할 수도 있다.

JOIN과 서브쿼리 선택 기준

상황	권장
양쪽 테이블 컬럼이 모두 필요	JOIN
존재 여부만 확인	EXISTS
집계값과 비교	서브쿼리
단계별 논리 전개	FROM 절 서브쿼리(파생 테이블)

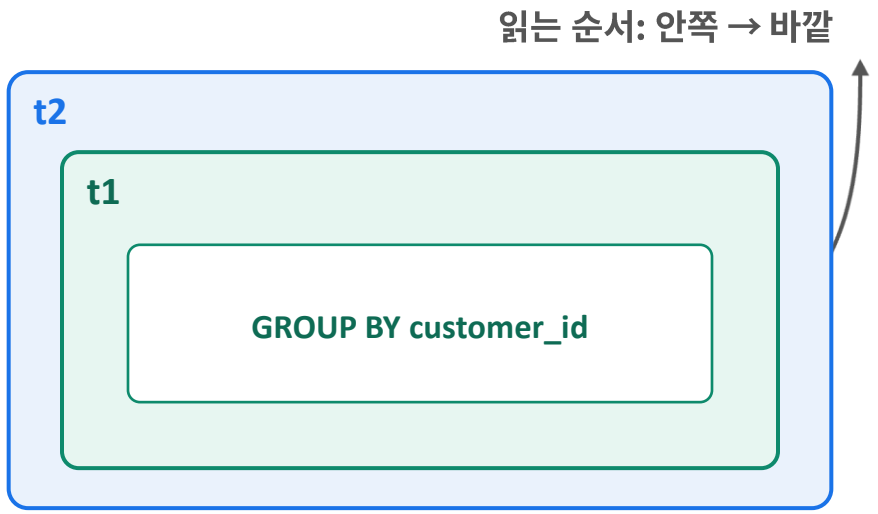
“단계별 논리 전개”는 **한 번의 계산으로 답이 안 나와서, 앞 단계의 결과를 받아 다음 단계를 계산해야 하는 문제**를 의미

서브쿼리 사용 시 주의사항

주의사항	설명
다중 행에 = 사용 금지	IN, ANY, ALL을 사용한다
NOT IN의 NULL 확인	NOT EXISTS로 대체를 권장한다
FROM 절 별칭	PostgreSQL 16부터 생략 가능하나 작성을 권장
스칼라 서브쿼리	1개 컬럼·최대 1행, 0행은 NULL, 2행 이상은 오류

중첩이 깊어지면

```
-- 안쪽부터 읽어야 해서 흐름이 거꾸로다
SELECT * FROM (
   SELECT * FROM (
      SELECT customer_id, COUNT(*) AS order_count -- 주문 건수
      FROM orders GROUP BY customer_id
   ) AS t1 WHERE order_count >= 5
) AS t2 ORDER BY order_count DESC, customer_id;
```



결과 73행 (주문 5건 이상 고객)

중첩된 SQL을 이해하거나 오류를 찾을 때는 가장 안쪽 SELECT부터 하나씩 따로 실행해 단계별 결과를 확인한다.

핵심 정리

1) 서브쿼리는 SQL 안에 포함된 또 다른 SELECT 문이다.
2) 비상관 서브쿼리는 먼저 값을 구해 바깥 쿼리에서 쓴다고 이해한다.
3) 단일 값은 비교 연산자, 여러 값 비교는 IN/ANY/ALL, 행 존재 확인은 EXISTS를 사용한다.
4) 다중 행 결과에 =를 쓰면 오류가 발생한다.
5) NOT IN은 오른쪽에 NULL이 있고 같은 비NULL 값이 발견되지 않으면 TRUE가 아니라 NULL(unknown)이 되어 WHERE에서 기대한 행이 사라질 수 있다.
6) 관련 행이 없는 것을 찾을 때는 NOT EXISTS나 LEFT JOIN + IS NULL을 쓴다.
7) EXISTS는 행의 존재 여부만 확인하며 SELECT 1을 관례로 쓴다.
8) 상관 서브쿼리는 메인쿼리의 값을 참조하므로 바깥 행마다 비교 기준이 달라진다.
9) FROM 절 서브쿼리(파생 테이블)의 별칭은 PostgreSQL 16부터 생략할 수 있지만, 컬럼 참조와 호환성을 위해 작성을 권장한다.
10) 스칼라 서브쿼리는 1개 컬럼·최대 1행을 반환하며, 0행이면 NULL이고 2행 이상이면 오류다.
11) 중첩이 깊어지면 가독성이 떨어지므로 안쪽 SELECT부터 단계별로 실행해 확인한다.